

Develop Role-Based UI Tests Using Stored Login States

Overview

You've explored how Playwright's browser contexts and authentication state storage can be used to simulate different users without logging in repeatedly. Now it's time to apply those concepts in a practical, role-based testing scenario.

Objective

This activity challenges you to apply Playwright's session storage and browser context features to simulate real-world role-based access control. You'll validate that different users see only what they should—without redundant logins. Are your tests enforcing the right access boundaries? This is your chance to scale secure, efficient role testing with confidence.

Scenario

You're working on a web application that supports multiple user roles—such as Regular User and Admin. Your goal is to validate that each role sees the correct UI elements and has access to only their permitted actions, without duplicating login steps across tests.

Your Mission

Simulate real-world role-based access control by:

- * Logging in once with each role

- * Saving their authentication state

- * Writing tests that verify each role's access to specific pages, buttons, or features

Instructions for learners

Task 1: Log In and Save Auth States**

→ 1. Use Playwright to log in once for both:

→ * A Regular User

→ * An Admin User

→ 2. Save their session using `context.storageState()`

```
```ts
// login-user.ts
import { chromium } from '@playwright/test';

(async () => {
 const browser = await chromium.launch();
 const context = await browser.newContext();
 const page = await context.newPage();
 await page.goto('https://your-app.com/login');
 await page.fill('#username', 'regular_user');
 await page.fill('#password', 'userpass123');
 await page.click('button[type="submit"]');
 await context.storageState({ path: 'user.json' });
 await browser.close();
})();
```
```

Repeat similarly for admin.json.

Task 2: Write Role-Based UI Tests

3. In your test file, create two tests using `browser.newContext({ storageState })`.

4. Verify that:

- * Regular Users can see only user-level features (e.g., "My Profile", "Submit Request")

- * Admins can see admin-level features (e.g., "User Management", "System Settings")

Task 2: Write Role-Based UI Tests

3. In your test file, create two tests using `browser.newContext({ storageState })`.

4. Verify that:

- * Regular Users can see only user-level features (e.g., "My Profile", "Submit Request")

- * Admins can see admin-level features (e.g., "User Management", "System Settings")

```
``ts
import { test, expect, chromium } from '@playwright/test';

test.describe('Role-Based Access Tests', () => {

test('Regular user sees limited dashboard features', async () => {
  const browser = await chromium.launch();
  const context = await browser.newContext({ storageState: 'user.json' });
  const page = await context.newPage();
  await page.goto('https://your-app.com/dashboard');
  await expect(page.getByText('My Profile')).toBeVisible();
  await expect(page.getByText('Submit Request')).toBeVisible();
  await expect(page.getByText('User Management')).not.toBeVisible();
  await browser.close();
});
```

```
test('Admin user sees all dashboard features', async () => {  
  const browser = await chromium.launch();  
  const context = await browser.newContext({ storageState: 'admin.json' });  
  const page = await context.newPage();  
  await page.goto('https://your-app.com/dashboard');  
  await expect(page.getByText('User Management')).toBeVisible();  
  await expect(page.getByText('System Settings')).toBeVisible();  
  await browser.close();  
});  
  
});  
````
```

## Completion

### Criteria

- \* Login states are saved and reused for both roles
- \* Tests use `browser.newContext({ storageState })` to simulate sessions
- \* Role-based feature access is validated with assertions
- \* Admin-specific features are restricted from regular users
- \* No login is repeated across tests

# Reflection

## Prompts

- \* How did using storageState improve test performance and structure?
- \* What UI elements were unique to each role in your app?
- \* How would this approach scale if your app had 4 or 5 roles?
- \* How can this test design help enforce access control across your team's QA process?